

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO4:

Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards
(BL2,BL6)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks:25

Annexure A

Coding challenge

Code of Conduct:

Shanika (192511387)

I Betty A. (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Betty

Annexure A

Coding challenge

1. Write a Python script that generates a star topology diagram (ASCII or graphical) given n hosts and a central switch. The script must print each host label, the switch name, and the connecting Cat6 UTP cable length.
2. Code a function `validate\_segment(length\_m)` that returns True if the UTP segment is within IEEE 802.3 limits (≤ 100 m) and False with an error message otherwise. Include at least 3 test cases.

3. Write a script that simulates CSMA/CD on a star topology with 4 hosts. Log every collision detection event and the back-off interval to a file named 'csma_log.txt'.
4. Using Python's 'socket' library, write a client-server program that models two hosts communicating through a star switch. Demonstrate message forwarding by printing the switch's MAC address table after each frame transmission.
5. Create a class 'StarSwitch' with methods 'add_port(host_id)', 'remove_port(host_id)', and 'broadcast(frame)'. Broadcast must send a frame to all ports except the source and log each delivery.

RUBRICS FOR CALCULATION

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Problem Understanding	15%	Fully understands the problem and requirements; no clarification needed	Understands most requirements with minor clarification	Partial understanding; misses some requirements	Poor understanding of the problem
Algorithm Design	20%	Efficient, optimal, and well-structured algorithm	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or no clear algorithm
Code Quality	15%	Clean, well-organized, readable, and properly formatted code	Mostly clean code with minor issues	Code is somewhat unclear or inconsistent	Poorly written, unreadable code
Functionality	20%	Code works perfectly for all test cases	Works for most test cases	Works for limited cases	Does not run or fails major cases
Error Handling	10%	Handles all edge cases and errors effectively	Handles some edge cases	Limited error handling	No error handling
Efficiency	10%	Highly optimized in time and space complexity	Reasonably efficient	Some inefficiencies	Highly inefficient
Documentation & Comments	5%	Clear and meaningful comments and documentation	Adequate comments	Few or unclear comments	No comments
Testing & Debugging	5%	Thoroughly tested with multiple cases; no bugs	Tested with some cases	Minimal testing	No testing done

1) Star Topology:

```
def star_topology(n):  
    switch = "SWITCH-1"  
    print("\n STAR TOPOLOGY")  
    print("    ", switch)  
    for i in range(1, n+1)  
        length = int(input(f"Enter  
            cable length for Host - {i} :"))  
        print(f"    {i}")  
        print(f"    {i} ... {length} m cable UTP  
            --- Host - {i}")  
    print("\n switch Name: ", switch)  
  
n = int(input("Enter number of hosts: "))  
star_topology(n)
```

Sample Output :

Enter number of hosts : 3

Enter ^{cable} ~~number~~ length for Host - 1 : 20

Enter cable length for Host - 2 : 30

Enter cable length for Host - 3 : 40

STAR TOPOLOGY

SWITCH - 1

1

1 --- 20m Cat6 UTP --- Host - 1

1 --- 30m Cat6 UTP --- Host - 2

1 --- 40m Cat6 UTP --- Host - 3

2. UTP Segment Validation:

```
def validate_segment(length-m):
```

```
    if 0 <= length-m <= 100:
```

```
        print(length-m, "m is a valid UTP  
segment.")
```

```
        return True
```

```
    else:
```

```
        print("Error: UTP length must  
not exceed 100m.")
```

```
        return False
```

```
validate_segment(50)
```

```
validate_segment(100)
```

```
validate_segment(120)
```

Sample Output:

50m is a valid UTP segment

100 m is a valid UDP segment

Error: UDP length must not exceed 100 m.

3. CSMA/CD Simulation:

```
import random
hosts = ["Host-1", "Host-2", "Host-3", "Host-4"]
with open("csma-log.txt", "w") as file:
    for i in range(5):
        sender = random.choice(hosts)
        if random.choice([True, False]):
            backoff = random.randint(1, 10)
            msg = f"collision at {sender}"
            BACK-OFF = f"{backoff} ms"
            print(msg)
            file.write(msg + "\n")
        else:
            msg = f"{sender} transmitted successfully"
            print(msg)
            file.write(msg + "\n")
    print("\n Log saved in csma-log.txt")
```

Sample Output :

Host - 1 transmitted successfully
collision at Host - 3, Back-off = 5ms
Host - 2 transmitted successfully
collision at Host - 4, Back-off = 3ms
Host - 1 transmitted successfully.
Log saved in csma-log.txt

4. Client - Server Using Socket :

Server program :

```
import socket
server = socket.socket()
server.bind(("localhost", 5000))
server.listen(1)
Print("Switch waiting for connection...")
client, address = server.accept()
Frame = client.recv(1024).decode()
Print("\nFrame received:", Frame)
print("Switch MAC Address Table:")
print("Host - 1 ->", address)
```



```
client.send(b"Frame forwarded successfully")
client.close()
server.close()
```

Client Program:

```
import socket

client = socket.socket()
client.connect(("localhost", 5000))

frame = "Host - 1, Host - 2, Hello Host - 2"
client.send(frame.encode())
response = client.recv(1024).decode()
print("server: ", response)
client.close()
```

Sample Output:

Client:

server: Frame forwarded successfully

Server:

Switch waiting for connection....

Frame received: Host - 1, Host - 2, Hello
Host - 2

Switch MAC Address Table:

Host - 1 → ('127.0.0.1', 54321)

5. starswitch class

```
class starswitch:
```

```
    def __init__(self):
```

```
        self.ports = []
```

```
    def add_port(self, host-id):
```

```
        self.ports.append(host-id)
```

```
        print(host-id, "connected")
```

```
    def remove_port(self, host-id):
```

```
        if host-id in self.ports:
```

```
            self.ports.remove(host-id)
```

```
            print(host-id, "removed")
```

```
    def broadcast(self, frame):
```

```
        source = frame["source"]
```

```
        print("In Broadcasting frame:", frame  
            ["data"])
```

```
        for host in self.ports:
```

```
            if host != source:
```

```
                print("Frame delivered to",  
                    host)
```

```
switch = starswitch()
```

```
switch.add_port("Host-1")
```

```
switch.add_port("Host-2")
```



```
switch.add-port ("Host-3")  
switch.add-port ("Host-4")
```

```
frame = {
```

```
  "source": "Host-1",
```

```
  "data": "Hello Hosts"
```

```
}
```

```
switch.broadcast(frame)
```

```
switch.remove-port ("Host-4")
```

Sample Output :

Host - 1 connected

Host - 2 connected

Host - 3 connected

Host - 4 connected

Broadcasting Frame : Hello Hosts

Frame delivered to Host - 2

Frame delivered to Host - 3

Frame delivered to Host - 4

Host - 4 removed

Q411